

课程笔记

Codex 中的 Plan Mode 与 update_plan: 计划动力学的设计与实现

五道口纳什

2026/04/09

Codex Plan Mode update_plan

视频作者/频道: 五道口纳什

发布日期: 2026/04/09

视频时长: 14:48

视频链接: <https://www.bilibili.com/video/BV1NdDtBjEg7>

目录

1 引言：为什么 Codex 需要 Plan	2
1.1 Codex 的完整开发流程：先 Exploration，再 Plan，最后 Execution	2
1.2 从 ReAct 与 Plan Agent 到 Codex：planning 是 coding agent 的关键设计	2
1.3 两种实现路径的预告：Plan Mode（对话式）与 update_plan（工具调用式）	3
1.4 本章小结	4
2 Plan Mode：通过对话协作生成计划	5
2.1 触发与上下文注入：/plan 命令与 develop 消息	5
2.2 协作模式的行为约束：非变更、探索、request_user_input	6
2.3 Plan Mode 的产物与退出机制：XML 封装的 Markdown 文本	8
2.4 本章小结	8
3 update_plan 工具：执行期的 Live Progress Model	8
3.1 工具定位：不是模式开关，而是执行期的 to-do / checklist 工具	8
3.2 Schema 解析：explanation、plan（list of steps）、status	9
3.3 状态机设计：pending / in_progress / completed	10
3.4 本章小结	12
4 Plan Dynamics：创建、更新与 Replan 的统一机制	12
4.1 单一函数，三种语义：create、update、replan	13
4.2 创建计划：首次调用时将首步置为 in_progress	13
4.3 状态更新与 Replan：提交完整快照，而非局部 patch	14
4.4 本章小结	14
5 工程视角：Codex 的通用工具系统设计	15
5.1 通用性与简洁性：一个 function 覆盖 plan 的全生命周期	15
5.2 设计来源：base_instructions_default.md 中的 plan 与 update_plan 指引	16
5.3 与 Claude Code 的对比：Codex 的 plan 设计为何更优雅	16
5.4 本章小结	17
6 总结与延伸	19
6.1 核心要点回顾：两种 plan 路径的差异与统一	19
6.2 结构化提炼：核心主张、机制与实践启示	19
6.3 可迁移经验：如何在自己的 agent 中设计 plan 工具	20
6.4 系列收尾与展望	20

1 引言：为什么 Codex 需要 Plan

在 Modern Agent 系列的第 17 期中，我们将目光聚焦于 OpenAI Codex 的计划动力学（plan dynamics）设计。Codex 作为当前最具代表性的 coding agent 之一，其工具系统的简洁性与统一性已在系列前作中有所涉及；而本期要补全的关键拼图，正是当时被略过的 `update_plan` 工具，以及与之相对的 `plan mode` 协作模式。理解这两者，有助于建立对 Codex 整体工作流的完整认知，也能为自主构建 agent 的开发者提供可直接迁移的设计范式。

1.1 Codex 的完整开发流程：先 Exploration，再 Plan，最后 Execution

Codex 在处理一个代码库级别的开发需求时，遵循一条清晰的三阶段流水线：

1. **Exploration（探索）**：模型首先对代码库进行密集的、代码级别的探查，包括阅读文件结构、关键模块、依赖关系等。
2. **Plan（计划）**：在充分理解上下文后，模型形成一份可执行的计划，明确需要完成的步骤与顺序。
3. **Execution（执行）**：按照既定计划实际修改代码、运行命令、验证结果。

与

Claude Code 等同类工具相比，Codex 的显著特征在于 **前置 exploration** 的明确化。Codex 并非在收到用户 query 后直接涌现计划，而是先通过系统性的代码探查为后续规划奠定事实基础。这种“先探索、后计划、再执行”的流水线，是其架构层面简洁性与可靠性的来源之一。

1.2 从 ReAct 与 Plan Agent 到 Codex：planning 是 coding agent 的关键设计

回顾本系列的前作：第二期介绍了 ReAct（Reasoning + Acting）范式，第三期则深入分析了 Plan Agent——即在执行之前先进行显式规划，并在执行过程中动态调整计划的工作机制。这些工作的共同结论可以概括为一句话：对于 coding agent 而言，“要有 plan”是一项关键设计。

Codex 继承了这一思想，但将其工程化到了极致。它并未把“计划”仅仅当作模型推理的文本产物，而是通过两种互补的机制将其纳入系统架构：

- **Plan Mode（计划模式）**：一种显式的协作模式，用户通过 `/plan` 命令进入，模型通过对话与用户共同澄清需求，最终生成一份结构化的计划。
- **update_plan（工具）**：一种隐式的函数调用机制，模型在执行期（default mode）自行决定是否调用该工具来创建、更新或重新规划任务进度。

这两种路径分别对应了“对话式规划”与“工具式规划”，构成了 Codex 计划设计的双轨体系。

¹ 视频画面时间区间：00:01:09

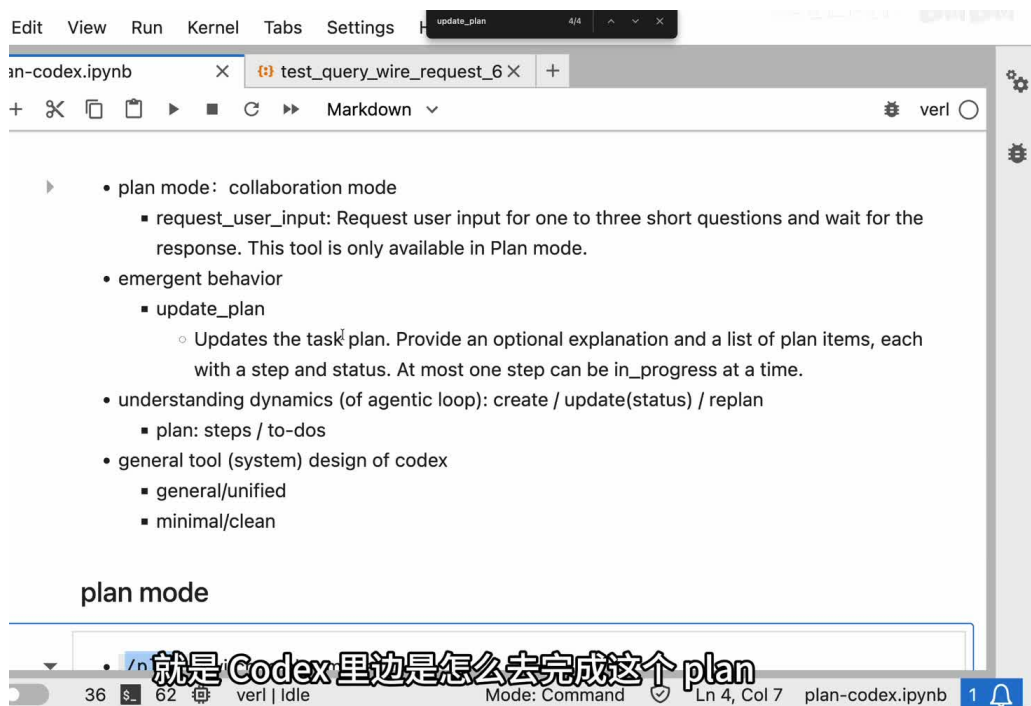


图 1: 本期核心议题概览: plan mode、update_plan、agent loop 中的计划动力学, 以及 Codex 的通用工具系统设计。¹

1.3 两种实现路径的预告: Plan Mode (对话式) 与 update_plan (工具调用式)

全文的论述将围绕这两条路径展开, 并揭示它们如何在统一的架构下协同工作。

Plan Mode 的本质是一种 **collaboration mode** (协作模式)。当用户在 Codex CLI 中输入 `/plan` 后, 系统会在 `develop` 消息的第二个 part 中注入一段冗长的 plan mode 专属 instruction。在此模式下, 模型不改代码, 仅通过 `request_user_input` 工具向用户发起结构化提问, 逐步聊出可直接实施的规格。最终产物是一份被 XML 封装的 Markdown 文本, **不落盘、不持久化**, 仅以字符串形式存在于 context window 中。当计划成型后, 模型会询问用户是否执行; 若用户确认, 则切换回 default mode 开始执行。

update_plan 则完全不同。它不是进入或退出 plan mode 的开关, 而是一个在执行期被模型自行涌现调用的 function tool。其 schema 极为简洁, 仅包含 **explanation** (解释为何有此计划) 和 **plan** (步骤列表, 每个步骤配一个 **status**) 两个核心字段。通过同一个函数、不同的传参方式, Codex 实现了计划的 **创建、状态更新与 replan (重新规划)**——这正是所谓 **plan dynamics (计划动力学)** 的核心内涵。

²视频画面时间区间: 00:05:12

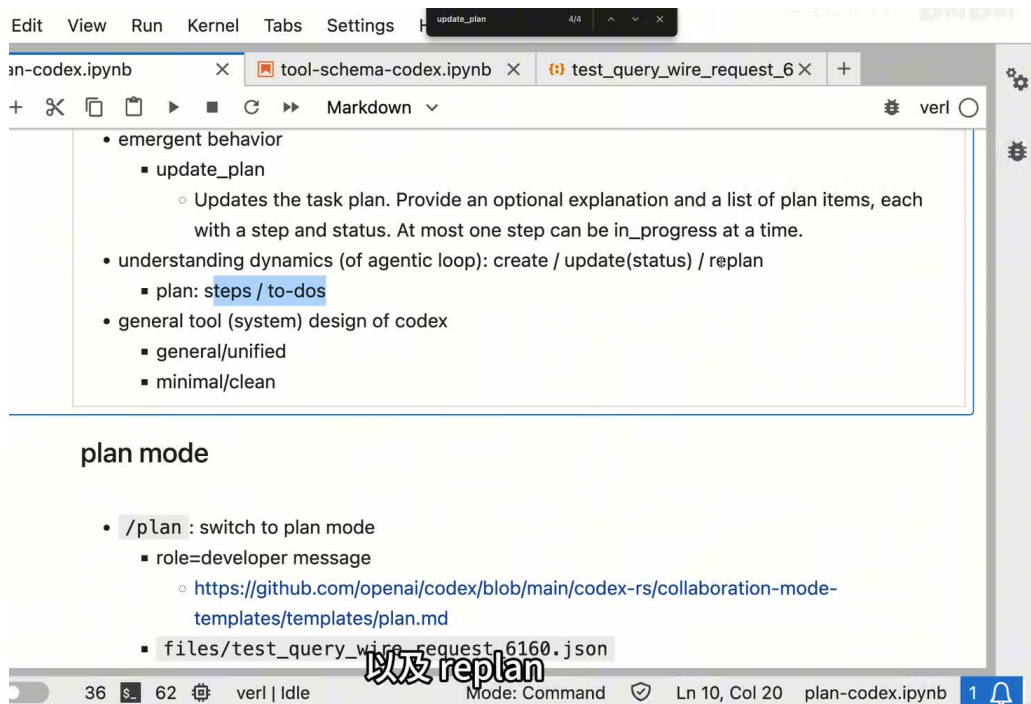


图 2: Codex plan mode 概览: `request_user_input` 仅在 plan mode 中可用, `update_plan` 则作为执行期的任务计划更新器, 二者分工明确。²

术语速览

- **plan mode**: 通过 `/plan` 显式进入的对话式协作规划模式。
- **update_plan**: Codex 的 function tool, 用于在执行期管理计划的全生命周期。
- **plan dynamics**: 计划动力学, 指计划在执行过程中的创建、更新与 `replan` 的动态行为。
- **replan**: 当执行中发现原计划不可行时, 通过 `update_plan` 提交全新计划快照的行为。
- **context window**: 模型可访问的当前对话及系统提示的文本范围, plan mode 的产物仅驻留于此。

1.4 本章小结

本章作为全文的引言, 建立了理解 Codex 计划设计的三个基本认知:

1. Codex 的完整开发流程遵循 **Exploration -> Plan -> Execution** 的三阶段流水线, 前置探索是其区别于其他 coding agent 的重要特征。
2. 从 ReAct 到 Plan Agent, 再到 Codex, “planning” 始终是 coding agent 架构中的关键设计要素。
3. Codex 通过两种互补机制实现规划: **Plan Mode** (对话式、显式、协作型) 与 **update_plan** (工具调用式、隐式、执行期动态涌现)。后续章节将分别深入解析这两种机制的实现细节与设计哲学。

2 Plan Mode: 通过对话协作生成计划

Codex 中生成与维护计划存在两条路径：一条是以 `/plan` 命令显式触发的 plan mode（计划模式），另一条是在 default mode（默认模式）下由模型自行涌现的 `update_plan` 工具调用。本节聚焦于前者，剖析其触发机制、行为约束与产物形态。

2.1 触发与上下文注入：/plan 命令与 develop 消息

用户在 Codex CLI 中输入 `/plan` 后，系统即切换至 plan mode。与常见的模式切换不同，Codex 并未在客户端维护一个独立的状态机，而是通过在请求体的 `input` 字段中注入一条 `role=developer` 的 develop 消息，向模型下达关于 plan mode 的完整行为约束。

具体而言，develop 消息通常包含多个 part，其中第二个 part 会嵌入一段冗长的 plan mode instruction。这段指令详细规定了模型在该模式下应当如何与用户协作、如何组织最终产物、以及何时退出该模式。图 3 展示了 `response.create` 请求体的基本结构，可见 `input` 列表中 `developer` 消息与 `user` 消息交错排列，而 plan mode 的切换完全依赖前者完成上下文注入。

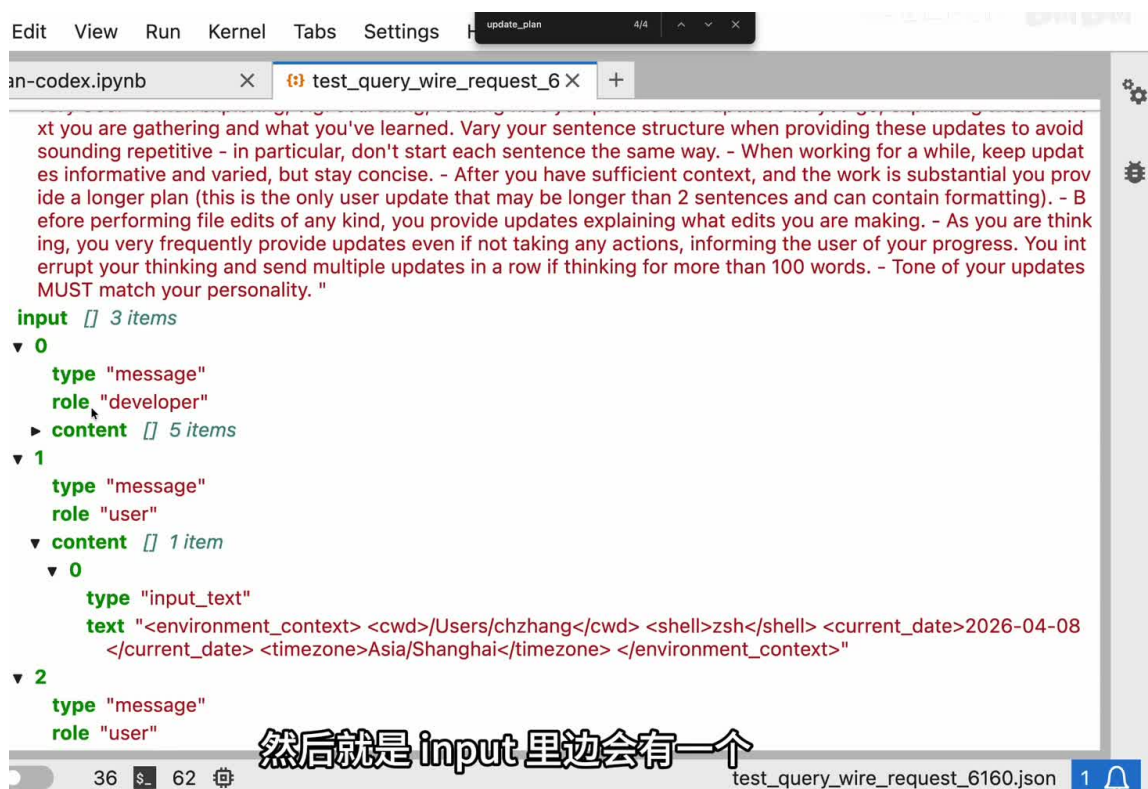


图 3: `response.create` 请求体的四个核心字段: `instructions`、`previous_response_id`、`input` 与 `tools`。其中 `input` 列表中的 `role=developer` 消息负责动态注入 plan mode 的行为约束。

P

lan mode 的切换并非客户端状态跳转，而是服务端提示工程（prompt engineering）的结果。系统通过在 develop 消息中追加模板化指令，使模型在单次请求内即获得完整的协作语义。这种设计体现了 Codex “context-engineering” 的核心思想：用消息编排替代硬编码状态机。

上述 develop 消息中引用的模板文件可在 OpenAI 官方仓库中查阅，路径为 `codex-rs/collaboration-mode-te`

此外，作者在本地保存了一份完整的请求体快照（`test_query_wire_request_6160.json`），供读者对照研究 plan mode 下 develop 消息的具体内容。

2.2 协作模式的行为约束：非变更、探索、request_user_input

Plan mode 本质上是一种 **collaboration mode**（协作模式）。其设计目标并非立即执行代码修改，而是通过与用户的对话“聊出一个可直接实施的规格”。因此，系统对模型在该模式下的行为施加了三条关键约束：

1. **非变更（non-destructive）**：在 plan mode 期间，模型不得对代码库进行任何实质性修改。所有操作仅限于阅读、分析与提问。
2. **探索（exploration）**：模型应主动澄清需求边界、确认技术方案，并在信息充分后才形成决策完备的计划。
3. **结构化提问**：系统推荐使用专用子工具 `request_user_input` 向用户发起结构化提问。该工具仅在 plan mode 下可用，支持一次性列出 1–3 个简短问题并等待用户回复，从而高效收敛需求。

图 4 与图 5 分别从概览与语义细节两个层面展示了 plan mode 的设计要点，包括 `request_user_input` 的可用性限制、`/plan` 切换机制，以及最终计划需封装于 `<proposed_plan>` XML 标签内的要求。

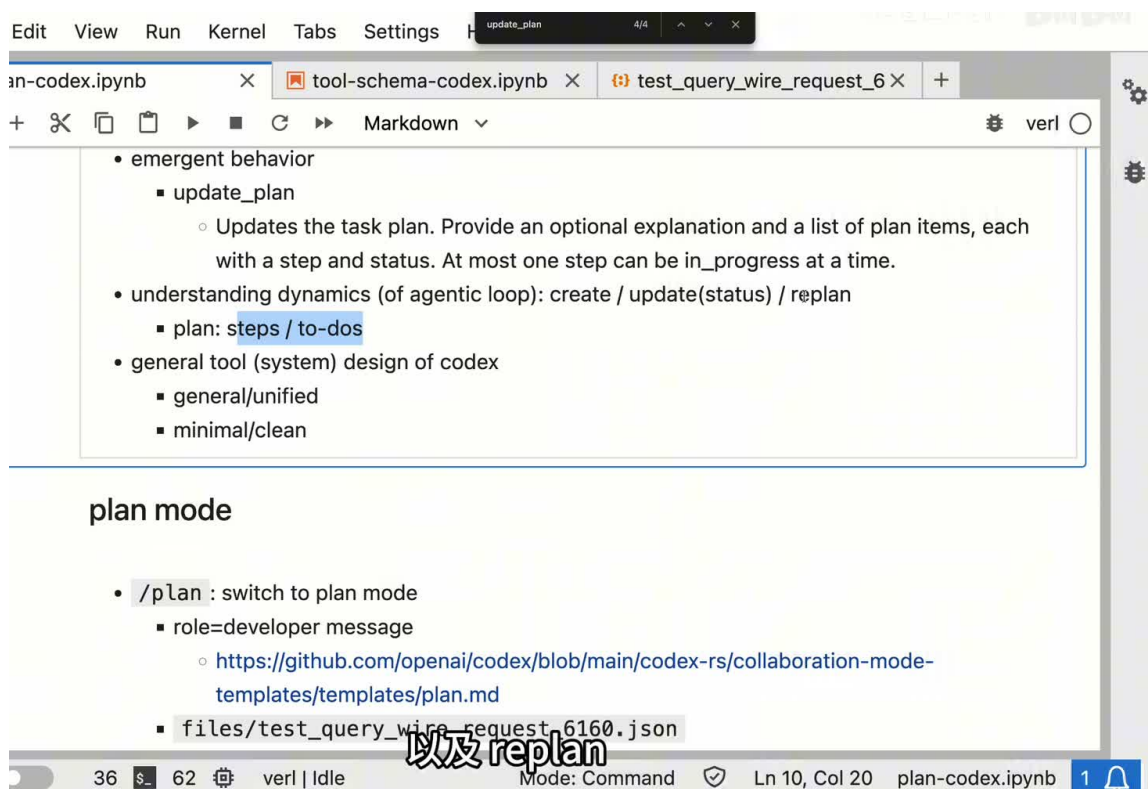


图 4: Plan mode 概览： `/plan` 命令触发 `role=developer` 消息注入，`request_user_input` 仅在 plan mode 下可用，而 `update_plan` 则作为任务计划更新器运行于执行期。

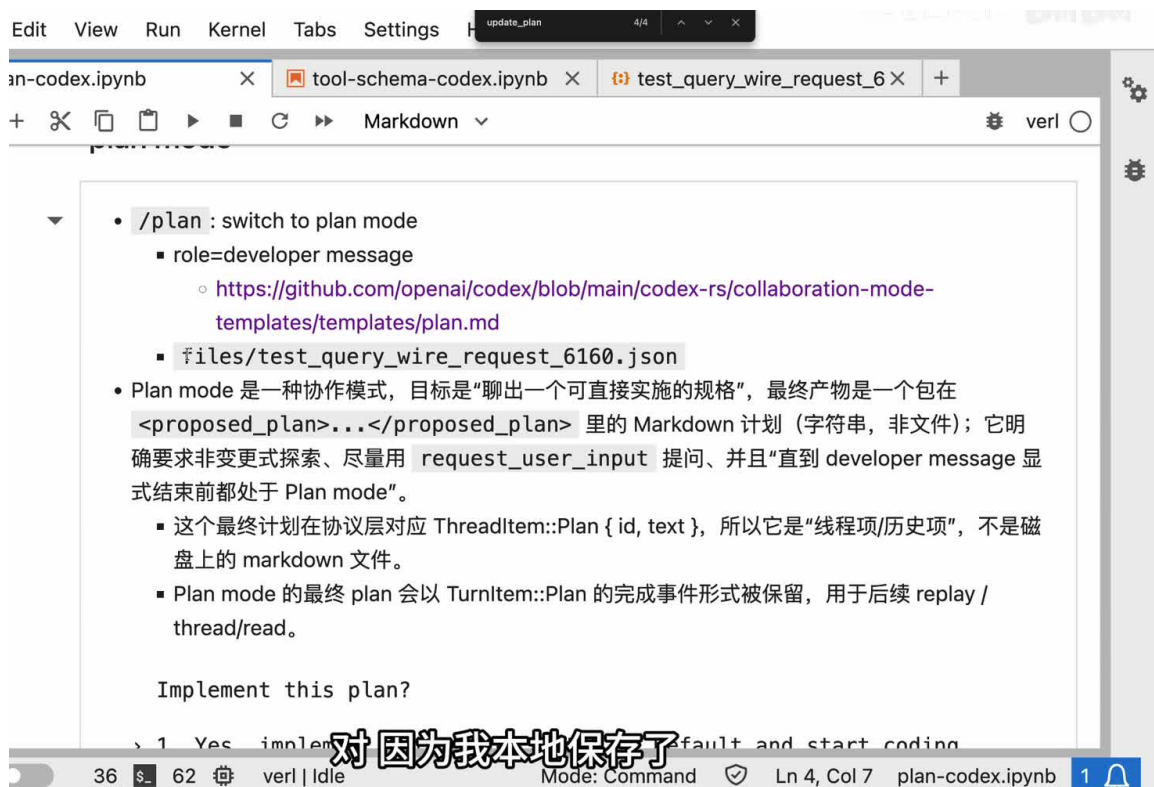


图 5: Plan mode 的详细语义：目标是在 `<proposed_plan>...</proposed_plan>` 中产出可直接实施的 Markdown 规格；明确要求非变更式探索，并尽量使用 `request_user_input` 进行结构化提问。最终计划在协议层对应 `TurnItem::Plan { id, text }`，用于后续 `replay / thread / read`。

request_user_input 的设计意图在于将”开放式闲聊”转化为”有边界的信息收集”。通过限制问题数量（1-3 个）并要求模型等待回复，该工具有效避免了 plan mode 中常见的过度发散或假设性推断，确保最终计划建立在用户确认的事实之上。

2.3 Plan Mode 的产物与退出机制：XML 封装的 Markdown 文本

当对话充分收敛后，模型需输出一份完整的计划。该计划并非持久化到本地的 Markdown 文件，而是以字符串形式存在于上下文窗口（context window）中：一段被 XML 标签 <proposed_plan> 与 </proposed_plan> 封装的 Markdown 文本。在协议层，这段文本会被映射为 TurnItem::Plan {id, text}，作为线程历史项保留，供后续 replay 或 thread read 使用。

Plan mode 的退出遵循显式确认机制：当计划生成完毕后，Codex 会向用户询问”是否要实现这个 plan？”（Implement this plan?）。若用户确认，系统即退出 plan mode，替换 develop 消息中的行为约束，恢复 default mode，并进入执行阶段。若用户拒绝或要求修改，则继续在 plan mode 中迭代，直至需求澄清完毕。

P

lan mode 的产物不落盘、不写入本地文件系统，其生命周期完全绑定于对话上下文。这意味着一旦开启新的对话线程或超出上下文窗口限制，该计划将无法自动恢复。对于需要长期维护的复杂需求，建议在确认计划后由用户自行保存关键内容。

2.4 本章小结

Plan mode 是 Codex 提供的一条显式规划路径。用户通过 /plan 命令触发后，系统以 develop 消息的形式向模型注入完整的协作语义；在该模式下，模型被约束为非变更探索，并优先使用 request_user_input 进行结构化提问；最终产物是一份以 XML 封装的 Markdown 计划字符串，经用户确认后退出 plan mode 并转入执行。理解这一机制，有助于把握 Codex 在”对话式规划”与”工具式规划”之间的架构分野，也为后续分析 update_plan 的执行期动力学奠定基础。

3 update_plan 工具：执行期的 Live Progress Model

在理解了 Plan Mode 的对话式规划机制之后，本节将目光转向 Codex 在执行期（execution phase）使用的另一套计划工具——update_plan。如果说 Plan Mode 是一种显式的、人机协作的”前置规划”，那么 update_plan 则是一种隐式的、由模型在执行过程中自行涌现的”动态进度模型”。它不需要用户显式切换模式，而是以 function calling 的形式，在 agent loop 内部实时维护一份 to-do list 或 checklist，从而驱动任务从探索走向完成。

3.1 工具定位：不是模式开关，而是执行期的 to-do / checklist 工具

首先需要澄清一个常见的误解：update_plan 并不是进入或退出 plan mode 的开关。在 Codex 的 13 个工具列表中（见图 6），update_plan 被归类为执行期工具，其语义是”更新当前任务计划与步骤状态”。

具体而言，update_plan 具有以下三个关键定位特征：

³截图来源：视频 03:33。

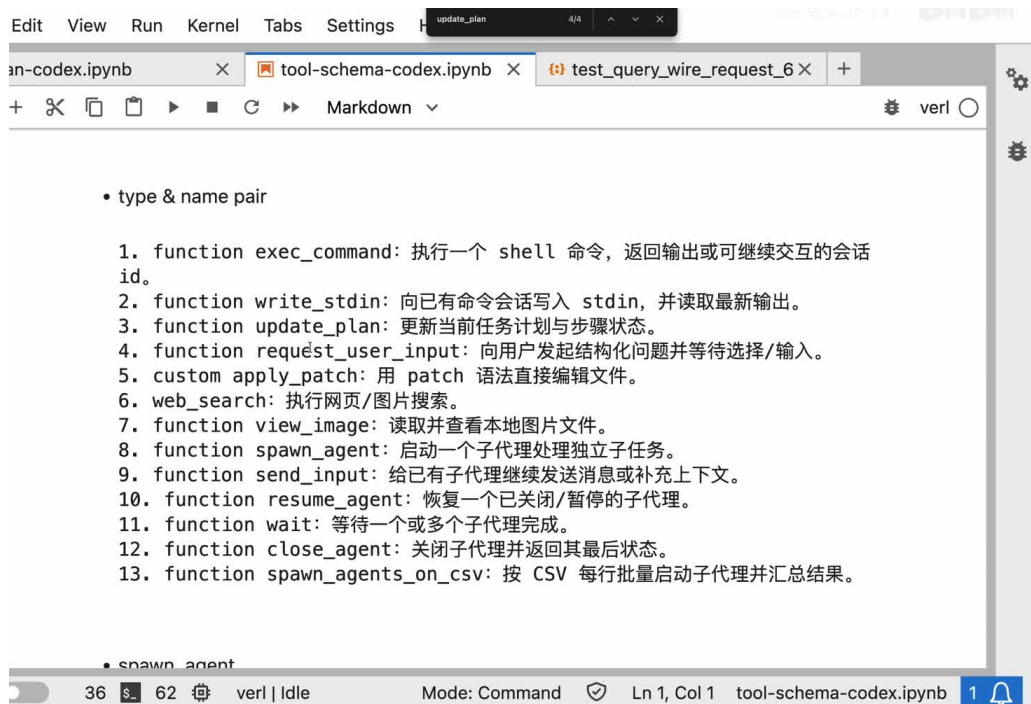


图 6: Codex 的 13 个工具枚举, `update_plan` 位列第 3, 语义为“更新当前任务计划与步骤状态”。

1. **执行期工具:** 它只在 default mode（常规执行模式）下被调用，用于向用户展示“当前任务进度”。如果在 plan mode 内部调用 `update_plan`，系统会直接报错。
2. **模型自行涌现:** 与 Plan Mode 需要用户通过 `/plan` 显式触发不同，`update_plan` 的调用是模型在执行过程中自发决定的。当模型认为有必要梳理剩余步骤、报告进度或调整策略时，它会主动发起该 function call。
3. **Live Progress Model:** 它本质上是一份对当前进度的内部表示（internal representation），不落盘、不持久化，仅存在于上下文窗口（context window）中，作为模型和用户共同可见的“实时进度条”。

核心认知

`update_plan` 不是模式切换器，而是执行期的 **TODO / checklist 工具**。它的价值在于让模型在 agent loop 中维持一份可更新的进度快照，从而以结构化方式推进复杂任务。

3.2 Schema 解析: explanation、plan (list of steps)、status

`update_plan` 的输入 schema 设计极为简洁，仅包含两个顶层字段: `explanation` 与 `plan`。图 7 展示了该工具在 notebook 中的文档定义。

以下是用 `lstlisting` 呈现的完整 JSON Schema 结构:

```
1 {
2   "explanation": "string_(optional)",
3   "plan": [
```

⁴截图来源: 视频 08:38。

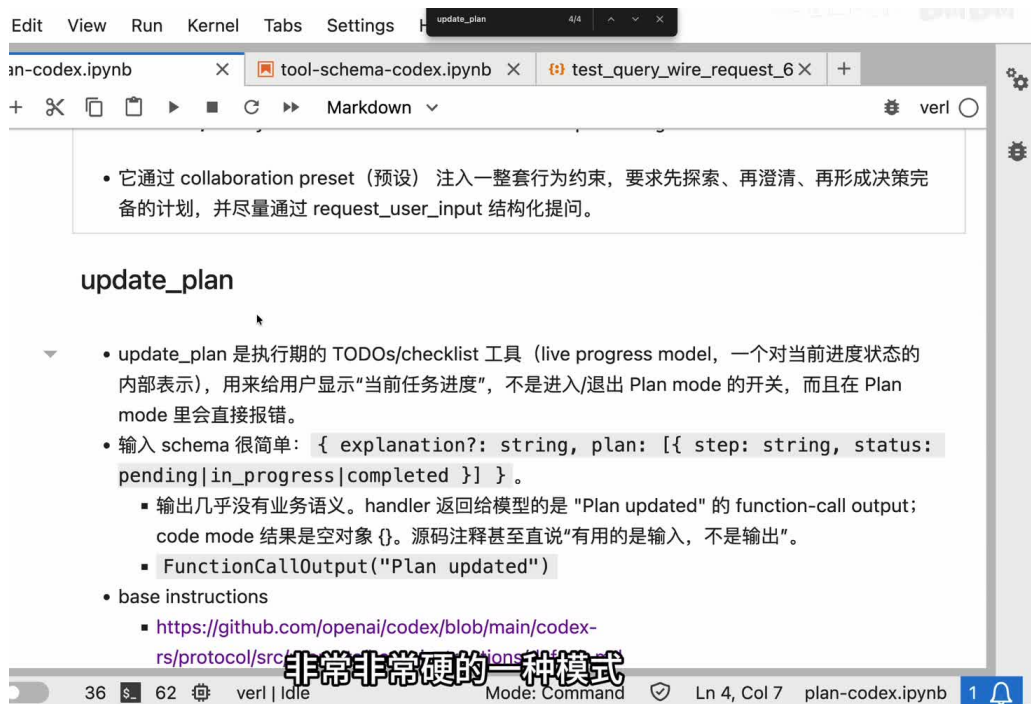


图 7: `update_plan` 工具的 schema 说明, 突出 `explanation`、`plan` 列表及 `FunctionCallOutput("Plan updated")` 的返回值语义。⁴

```

4  {
5      "step": "string",
6      "status": "pending|in_progress|completed"
7  }
8  ]
9  }

```

Listing 1: `update_plan` 的输入 Schema

各字段的语义如下:

- **explanation** (可选): 模型解释为何要创建或更新该计划。在 `replan` 场景中, 该字段尤为重要, 因为它需要向用户说明“原计划为何不可行”以及“新计划的调整逻辑”。
- **plan** (必填): 一个对象列表, 每个对象包含:
 - **step**: 步骤描述, 通常是一句可执行的动作说明;
 - **status**: 该步骤的当前状态, 取值见下一小节。

值得注意的是, `update_plan` 的输出几乎没有业务语义。handler 返回给模型的是一个固定的 `function-call output`——`FunctionCallOutput("Plan updated")`, 在 `code mode` 下甚至只是一个空对象 `{}`。源码注释直截了当地指出: “有用的是输入, 不是输出”。这意味着该工具的核心价值完全体现在模型通过参数提交的 `plan` 快照上, 而非执行返回值。

3.3 状态机设计: `pending` / `in_progress` / `completed`

每个 `step` 的 `status` 只能从以下三个值中选取, 构成了一个极简但足够表达执行进度的状态机:

- **pending**: 步骤尚未开始，处于等待状态；
- **in_progress**: 步骤正在执行中；
- **completed**: 步骤已完成。

图 8 清晰展示了这三个状态值以及同 turn 内的更新规则。

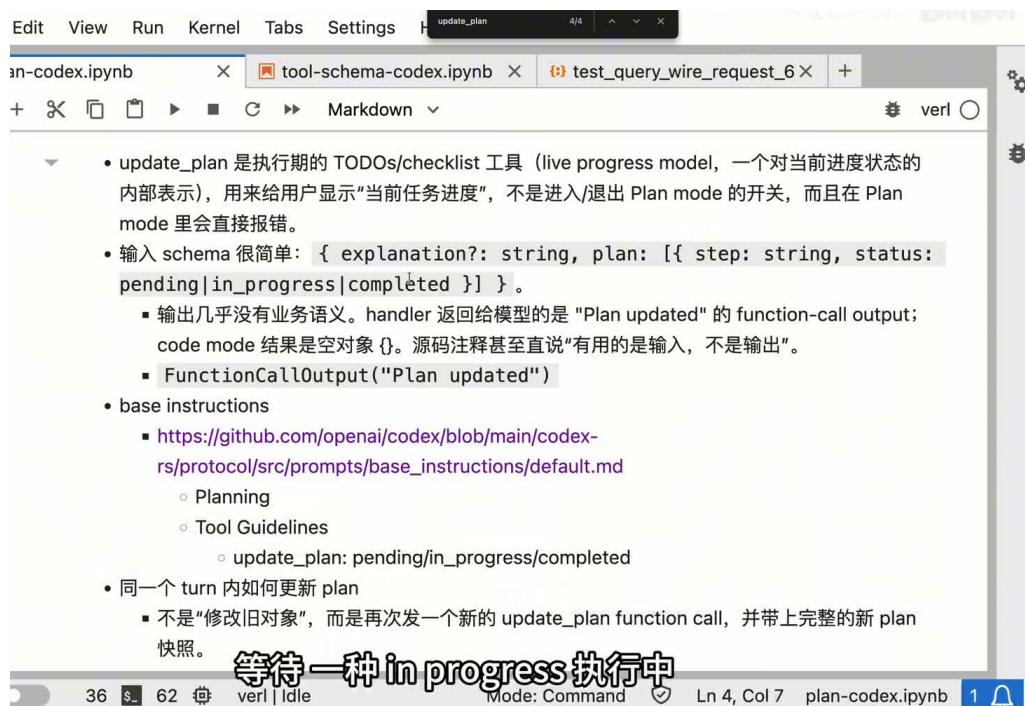


图 8: `update_plan` 的三个状态值 (`pending`、`in_progress`、`completed`) 以及“同 turn 内以完整快照替换”的更新规则。⁵

该状态机的设计遵循两条关键约束：

1. **单步执行约束**: 同一时刻，整个 plan 列表中最多只有一个 step 可以处于 `in_progress` 状态。这强制模型以串行、step-by-step 的方式推进任务，避免多步骤并发导致的逻辑混乱。
2. **快照替换约束**: 在同一个 turn 内更新 plan 时，不是对旧对象做局部 patch（例如只改某一个 step 的 status），而是再次发起一次全新的 `update_plan` function call，并附上完整的、最新的 plan 快照。

图 9 以流程图形式概括了 `update_plan` 在 agent loop 中的动态行为：首次调用即视为“创建计划”，自然将第一个 step 置为 `in_progress`；随后通过反复调用更新状态；当执行遇阻时，可触发 `replan`，提交一份全新的 plan 快照，从而进入反馈循环。

⁵截图来源：视频 09:24。

Codex Agent Loop Dynamics

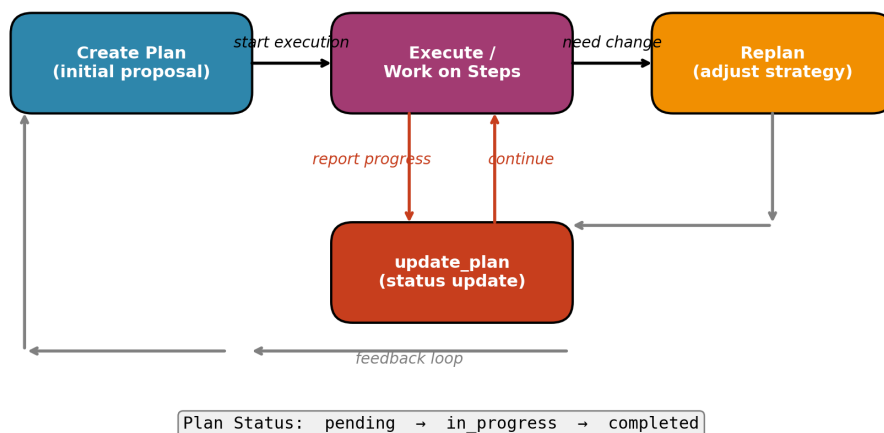


图 9: Codex agent loop 动力学示意图，展示 `update_plan` 如何在计划创建、执行、状态报告与 `replan` 之间形成闭环。

高信号要点

- `update_plan` 的返回值固定为 "Plan updated"，业务语义完全承载于输入参数。
- 最多只有一个 step 处于 `in_progress`，确保执行顺序清晰可控。
- 更新机制是完整快照替换，而非局部 patch，这为后续 `replan` 提供了统一的数据结构基础。

3.4 本章小结

本章深入解析了 `update_plan` 作为 Codex 执行期核心工具的设计理念。与 Plan Mode 的对话式、前置式规划不同，`update_plan` 是一种嵌入在 agent loop 内部的 live progress model:

- 它不是模式开关，而是模型在执行过程中自行涌现的 **TODO / checklist 工具**；
- 其 schema 极其简洁，仅由 `explanation` 和 `plan`（含 `step` 与 `status`）构成；
- 通过 `pending`、`in_progress`、`completed` 三种状态，配合“单步执行”与“快照替换”两条约束，实现了对复杂任务的串行推进与动态跟踪。

这一设计为下一章探讨 `update_plan` 如何以单一函数统一承载 `create`、`update`、`replan` 三种语义奠定了基础。

4 Plan Dynamics: 创建、更新与 Replan 的统一机制

在理解了 `update_plan` 的 schema 与状态机之后，本节进一步揭示 Codex 在工具系统设计层面的核心思想：通过单一函数 `update_plan`，以参数变化而非 API 增殖的方式，覆盖计划的全生命

周期管理——包括首次创建、执行中的状态更新、以及遇到阻碍时的重新规划（replan）。这种最小化、统一化的接口设计，是 Codex 通用工具系统（general tool system design）的典型体现。

C

odex 的 `update_plan` 没有 `create_plan` 与 `update_plan_status` 之分。计划的创建、状态推进与 `replan`，全部通过向同一函数提交不同参数的完整快照来完成。这是其工具接口“极简而统一”设计哲学的集中体现。

4.1 单一函数，三种语义：create、update、replan

在传统工程思维中，针对计划的生命周期通常会设计多个 API：一个用于初始化计划（`create`），一个用于修改某一步的状态（`update`），再有一个用于整体替换计划（`replan`）。然而 Codex 选择了一条截然不同的路径——它只暴露一个函数 `update_plan`，而让这个函数具备三种语义：

1. **Create（创建）**：当模型在某一 turn 内首次调用 `update_plan` 时，即视为向系统宣告“我现在要建立一个计划”。此时传入的 `plan` 列表即为初始计划。
2. **Update（更新）**：在执行过程中，模型再次调用 `update_plan`，传入与上一次结构相同但状态已推进的 `plan` 列表，从而更新当前进度。
3. **Replan（重新规划）**：当执行中发现原计划的某一步不可行、或需要新增/删除步骤时，模型同样调用 `update_plan`，但传入的是一份全新的 `plan` 快照，以此完成整体替换。

这三种行为在调用层面没有任何区分标志——没有额外的模式参数，也没有不同的 endpoint。区分它们的唯一依据是调用时传入的 `plan` 内容相对于上下文中已有计划的关系：首次出现即为创建，结构不变仅状态改变即为更新，结构发生替换即为 `replan`。

4.2 创建计划：首次调用时将首步置为 `in_progress`

在 agent loop 的某个 turn 中，模型经过前期探索（`exploration`）后，决定进入有计划的执行阶段。此时它会发起第一次 `update_plan` 调用，其参数大致如下：

```
1 {
2   "explanation": "将任务拆分为三步：读取配置文件、修改对应字段、验证输出结果。",
3   "plan": [
4     {"step": "读取并解析配置文件", "status": "in_progress"},
5     {"step": "修改目标字段值", "status": "pending"},
6     {"step": "运行测试验证结果", "status": "pending"}
7   ]
8 }
```

Listing 2: 首次调用 `update_plan` 创建计划示例

值得注意的是，Codex 的约定要求：同一时刻最多只有一个 step 处于 `in_progress` 状态。因此，模型在首次创建计划时，会自然而然地将第一个 step 标记为 `in_progress`，其余步骤保持 `pending`。这不仅是状态机的约束，也反映了执行策略——串行推进、逐步完成。

4.3 状态更新与 Replan: 提交完整快照, 而非局部 patch

当第一个 step 执行完毕（例如文件已读取），模型需要推进到下一步。此时它不会调用某个虚构的 `patch_plan_status` 函数，而是再次发起一次完整的 `update_plan` 调用，并附上全新的计划快照：

```
1 {
2   "explanation": "配置文件已读取，接下来修改目标字段。",
3   "plan": [
4     {"step": "读取并解析配置文件", "status": "completed"},
5     {"step": "修改目标字段值", "status": "in_progress"},
6     {"step": "运行测试验证结果", "status": "pending"}
7   ]
8 }
```

Listing 3: 状态更新：提交新的完整快照

这种“快照替换”机制意味着：

- 系统层面无需实现复杂的增量合并（patch/merge）逻辑；
- 模型拥有对计划结构的完全控制权，可以随时增删步骤、调整顺序；
- 每次调用都是自包含的（self-contained），便于在上下文窗口中追踪历史演变。

更进一步，当执行到某一步时发现原计划不可行——例如配置文件格式与预期不符，需要新增一个“格式转换”步骤——模型便会触发 **replan**：

```
1 {
2   "explanation": "发现配置文件为 YAML 格式，需先转换为 JSON 再解析。",
3   "plan": [
4     {"step": "读取配置文件", "status": "completed"},
5     {"step": "将 YAML 转换为 JSON", "status": "in_progress"},
6     {"step": "修改目标字段值", "status": "pending"},
7     {"step": "运行测试验证结果", "status": "pending"}
8   ]
9 }
```

Listing 4: Replan: 替换整个计划结构

可以看到，`replan` 与普通的 `status update` 在调用形式上完全一致，区别仅在于 `plan` 列表的内容是否发生了结构性变化。这正是 Codex 设计的高明之处：用统一的调用模式，消解了 **create/update/replan** 的边界，让模型在单个 turn 内能够丝滑地完成计划的动态演进。

图 10 以可视化的方式总结了上述过程。

4.4 本章小结

本章深入剖析了 Codex 中 `update_plan` 的统一动力学机制。核心结论可归纳如下：

- 一函多义：`update_plan` 通过参数差异承载 `create`、`update`、`replan` 三种语义，无需拆分 API。
- 首步即执行：首次调用创建计划时，第一个 step 自动置为 `in_progress`，体现串行执行策略。

Codex Agent Loop Dynamics

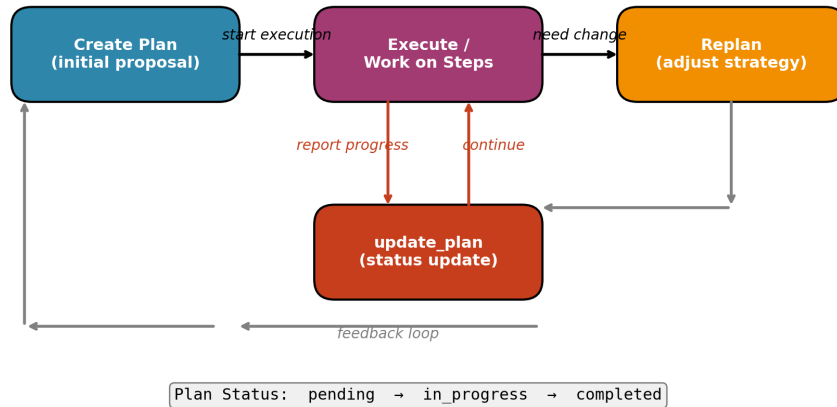


图 10: Codex agent loop 中的 plan dynamics: 通过 `update_plan` 实现计划的创建、状态推进与 replan 的闭环。

- **快照哲学:** 所有变更均以完整 plan 快照的形式提交，而非局部 patch；这降低了系统复杂度，同时赋予模型最大的灵活性。
- **设计意图:** 这种最小化、统一化的接口设计，是 Codex 通用工具系统设计的缩影，也是其与更复杂多 API 方案相比更显优雅的关键所在。

在下一章中，我们将把视角从 `update_plan` 本身拉高到 Codex 的整体工具系统，进一步探讨这种 “general tool system design” 的宏观设计原则，并与 Claude Code 的实现路径进行对比。

5 工程视角：Codex 的通用工具系统设计

本章核心观点

Codex 的 `update_plan` 并非孤立的”功能补丁”，而是其”通用工具系统 (general tool system design)” 设计哲学的集中体现：以最小、最统一的接口覆盖复杂行为的完整生命周期。

5.1 通用性与简洁性：一个 function 覆盖 plan 的全生命周期

在工程实现上，Codex 对计划动力学的处理展现出一种极致的极简主义：它没有为 plan 的创建、状态更新、重新规划分别设计三个 API，而是将这三种语义全部收敛到单一的 `update_plan` 函数调用中。Speaker 在 05:40 处明确指出，这种设计 “非常非常 general、非常非常 unified”，一个 function 即可实现 plan 的创建、每个 step 状态的更新以及 replan。

这种 “单函数全生命周期” 模式的优势体现在三个层面：

1. **接口最小化:** 工具 schema 仅需 `explanation`、`plan` (list of steps with status) 两个核心字段，降低了系统实现与维护成本；

2. **语义统一**: 无论首次创建还是中途 replan, 模型的行为模式完全一致——发起一次 `update_plan` function call 并提交完整的 plan 快照;
3. **状态机内聚**: `pending` $\rightarrow$ `in_progress` $\rightarrow$ `completed` 的状态推进完全由同一入口驱动, 避免了多 API 之间的状态同步问题。

如图 11 所示, 整个 agent loop 可以抽象为“创建计划 $\rightarrow$ 执行并更新状态 $\rightarrow$ 必要时 replan”的闭环, 而 `update_plan` 正是贯穿这一闭环的唯一工具节点。

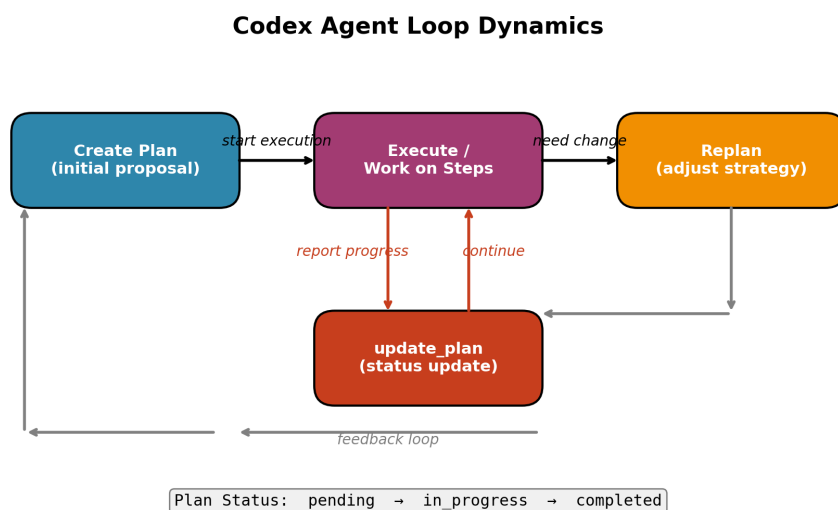


图 11: Codex agent loop 动力学示意图, `update_plan` 作为计划创建、状态更新与 replan 的统一工具节点。

6

5.2 设计来源: `base_instructions_default.md` 中的 plan 与 `update_plan` 指引

Codex 的 plan 设计并非凭空涌现, 其初衷可以在官方仓库的 `base_instructions_default.md` 中找到明确描述。Speaker 在 10:10 处展示了该文件 (见图 12), 并指出其中包含了关于 `update_plan` 的详细指引。

需要特别说明的是, 这些 base instructions 的内容未必以明文形式出现在每一次远端 API 请求体中, 但从语义层面看, 它们构成了 Codex 设计 `update_plan` 的底层逻辑来源。Speaker 进一步观察到, GPT-4 系列模型已经对这一工具的使用表现得“非常非常成熟”——这意味着: 当基础模型能力足够强时, 通过精心撰写的系统指令和一个监督式的工具参数设计, 即可在单个 turn 内丝滑地完成计划的创建、追踪与调整, 而无需引入复杂的外部编排框架。

5.3 与 Claude Code 的对比: Codex 的 plan 设计为何更优雅

Speaker 在 11:20 至 12:50 的段落中, 将 Codex 与 Claude Code 进行了直接对比。两者的核心差异可归纳如下:

⁶视频来源: 09:10–12:50, 对应 speaker 对 general tool system design 及 plan dynamics 的总结。

⁷视频来源: 10:37。

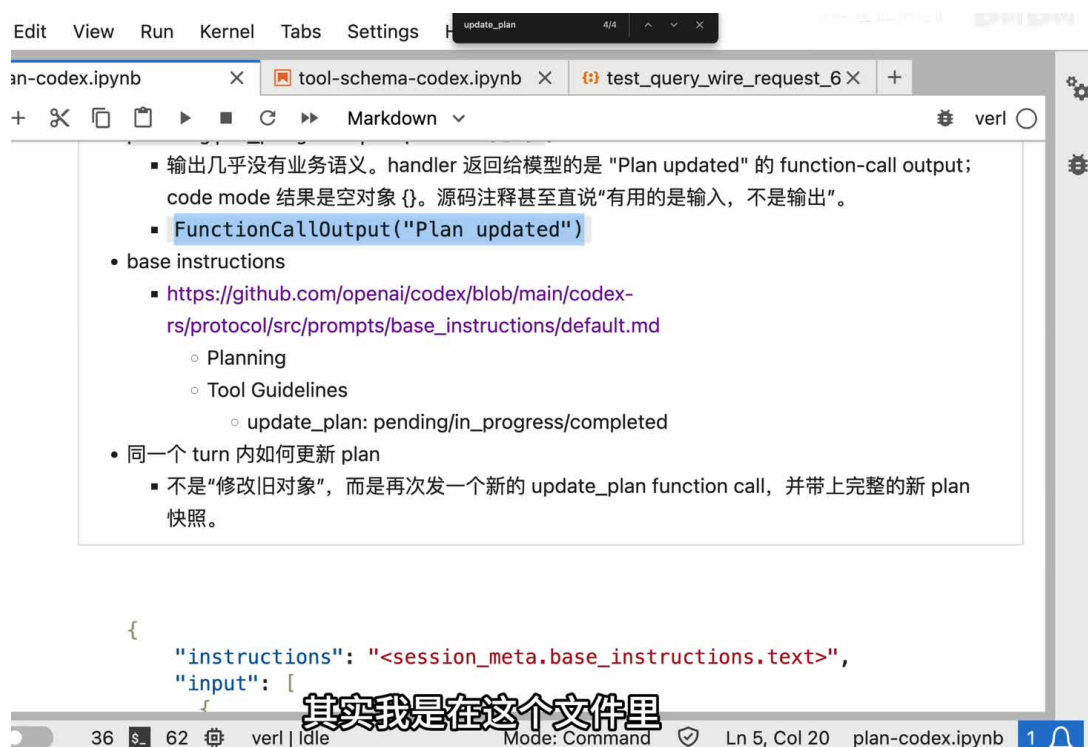


图 12: OpenAI Codex 官方仓库中的 `base_instructions_default.md`, 包含 `personality` 与 `AGENTS.md spec` 等核心行为约束。

7

- **Exploration 前置:** Codex 在执行计划前会先做密集的代码库级别 exploration, 而 Claude Code 的 plan 更多是模型在对话中自行涌现;
- **Plan 的显式化:** Codex 通过 `/plan` 模式显式产出可实施的规格文档, 并通过 `update_plan` 在执行期显式维护进度; Claude Code 则缺乏如此结构化的计划工具;
- **架构简洁性:** Codex 用单一函数覆盖 plan 的全生命周期, 而 Claude Code 的计划行为分散在模型的自然语言输出与多次交互中, 难以被外部系统观测或干预。

从工程视角看, Codex 的设计体现了一种“架构层面的优雅”: 它将 plan dynamics 从隐式的模型涌现转化为显式的、可观测的、可重放的工具调用序列。这种显式化不仅提升了系统的可控性, 也为后续的日志审计、错误回溯、人机协作提供了清晰的抓手。

设计启示

当模型能力达到 GPT-4 级别时, agent 系统的复杂度不应通过增加 API 数量来解决, 而应通过更精妙的指令工程 (instruction engineering) 和更内聚的工具语义来收敛。Codex 的 `update_plan` 正是这一思想的典范。

5.4 本章小结

本章从工程视角审视了 Codex 的通用工具系统设计。`update_plan` 以其“单函数、多语义、全快照”的极简设计, 实现了 plan 的创建、更新与 replan 的统一管理。其设计初衷可追溯至 `base_instructions_default` 中的系统指令, 而 GPT-4 系列模型对该工具的成熟运用, 证明了“强模型 + 精指令 + 内聚工具”

这一范式的可行性。与 Claude Code 相比，Codex 通过前置 exploration 与显式工具调用，将 plan dynamics 从隐式涌现提升为可观测、可控制的系统行为，展现出更高的架构简洁性与工程优雅性。

6 总结与延伸

本节定位

作为全文的最终收束，本节综合 speaker 的实质性结尾讨论，给出结构化的认知提炼、可迁移的工程经验，以及对 Modern Agent 系列的阶段性展望。

6.1 核心要点回顾：两种 plan 路径的差异与统一

回顾全文，Codex 提供了两种互补的 plan 机制：

1. **Plan Mode（对话式规划）**：通过 `/plan` 命令进入协作模式，利用 `request_user_input` 与用户结构化对话，最终产出 XML 封装的 Markdown 计划文本。该计划不落盘、不持久化，仅作为上下文窗口中的 `TurnItem::Plan` 存在，适用于需求澄清与规格定义阶段。
2. **update_plan（工具式规划）**：在执行期由模型自行涌现调用，以 `explanation + plan (steps with status)` 的极简 schema 维护实时进度。同一 turn 内通过多次 function call 实现状态推进与 replan，适用于任务执行与动态调整阶段。

这两种路径的共同点是“上下文内管理”——无论是 Plan Mode 的 Markdown 文本还是 `update_plan` 的 checklist，都不依赖外部持久化存储，而是直接在模型的 context window 中流转。这种设计降低了系统耦合度，同时也对上下文工程（context engineering）提出了更高要求。

6.2 结构化提炼：核心主张、机制与实践启示

基于 speaker 在 12:50 至 14:10 的总结，可将本期内容提炼为以下三个层次：

核心主张：对于 coding agent 而言，planning 不是可选装饰，而是关键设计。Codex 通过“前置 exploration + 显式 plan 机制 + 执行期动态更新”，构建了一个完整且优雅的计划动力学闭环。

核心机制：

- **单函数生命周期管理**：`update_plan` 以完整快照（snapshot）替代局部 patch，统一了 create、update、replan 三种语义；
- **状态机约束**：`pending/in_progress/completed` 三态设计，配合“同一时刻仅一个 step 处于 in_progress”的约束，保证了执行进度的清晰可追踪；
- **指令驱动涌现**：模型行为不依赖硬编码的编排逻辑，而是通过在 `base_instructions_default.md` 中的精细指令引导自然涌现。

实践启示：

- 在设计 agent 工具时，优先追求接口的“语义内聚”而非功能堆砌；
- 当模型能力足够强时，可通过“监督式参数 + 精写指令”替代复杂的外部状态机或工作流引擎；
- 计划的显式化（explicit plan representation）是提升 agent 可解释性与可控性的关键杠杆。

6.3 可迁移经验：如何在自己的 agent 中设计 plan 工具

Speaker 在 13:30 处特别强调了 `update_plan` 的可迁移价值。若要在自己的 agent 系统中实现类似的 plan dynamics，可参考以下设计 checklist：

Plan 工具设计 Checklist

1. **单一入口**：用一个 function 覆盖 plan 的创建、更新与 replan，避免 API 碎片化；
2. **完整快照**：每次更新都提交整个 plan 列表，而非局部 patch，确保上下文自洽；
3. **状态三态**：采用 `pending/in_progress/completed` 的简洁状态机，并限制同时仅有一个 `in_progress`；
4. **解释字段**：强制或鼓励提供 `explanation`，增强计划变更的可解释性；
5. **指令先行**：在系统提示词中详细描述工具的使用场景、调用时机与约束规则，让强模型自然涌现正确行为。

6.4 系列收尾与展望

Speaker 在 14:10 处宣告，本期是 Codex / Claude Code 系列的“暂告段落”——除非未来出现重大的底层架构更新或全新的概念原理引入。这一收尾并非意味着话题的终结，而是标志着一个认知阶段的完成：从 ReAct 到 Plan Agent，再到 Codex 的 Plan Mode 与 `update_plan`，我们已经见证了 coding agent 中“计划”这一核心能力从隐式涌现走向显式工具化、从外部编排走向指令驱动内聚的完整演进。

展望未来，随着基础模型能力的持续提升，agent 系统的设计重心将越来越从“复杂的控制流工程”转向“优雅的上下文与工具语义设计”。Codex 的 `update_plan` 所代表的“最小工具、最大覆盖”的通用工具系统思想，或将成为下一代 agent 架构的重要参照范式。